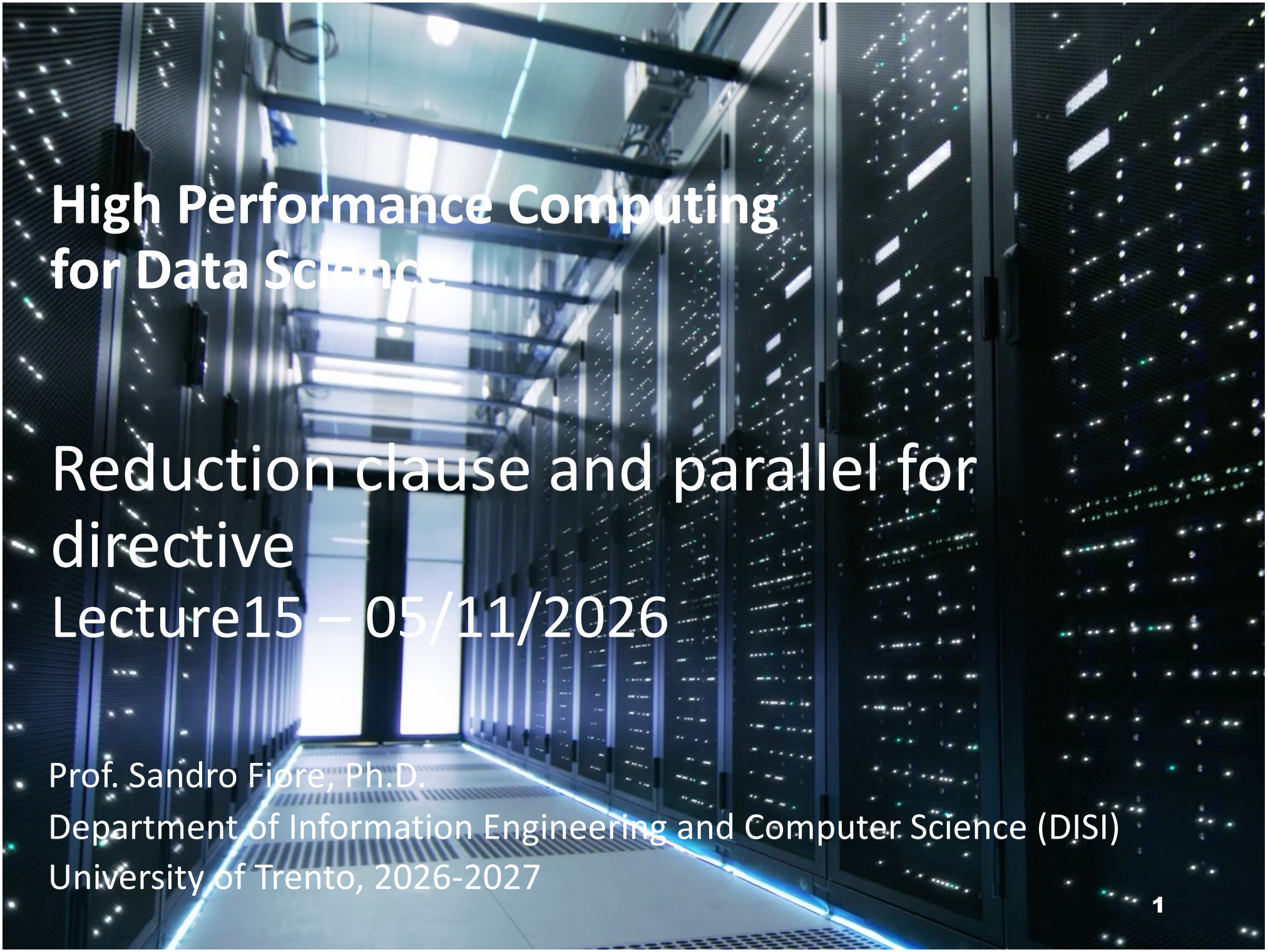


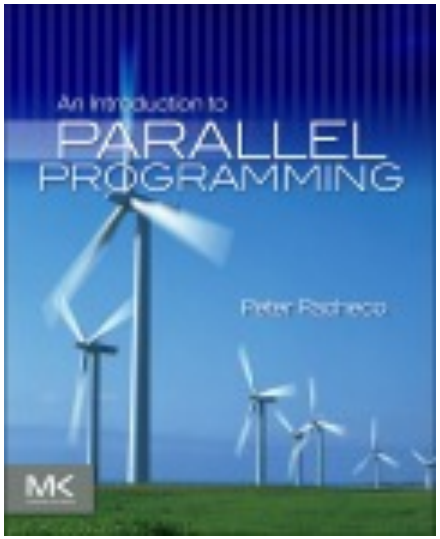
High Performance Computing for Data Science

Reduction clause and parallel for directive
Lecture15 – 05/11/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Outline and material

- Introduction to OpenMP (recap)
- The trapezoidal rule (recap)
- The reduction clause
- The parallel for directive



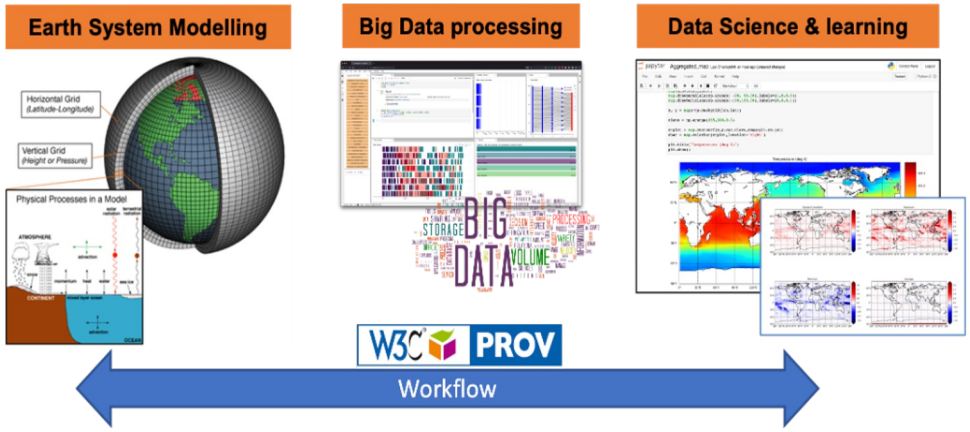
Material

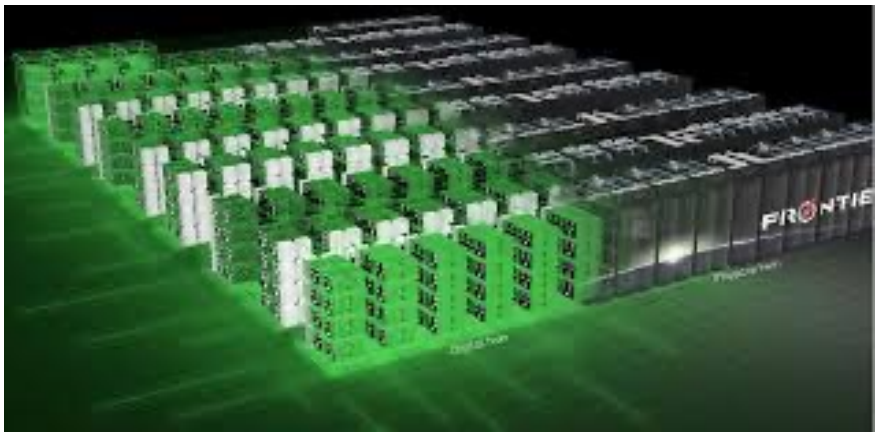
Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 5
Shared Memory Programming with OpenMP

Seminar dates (confirmed)

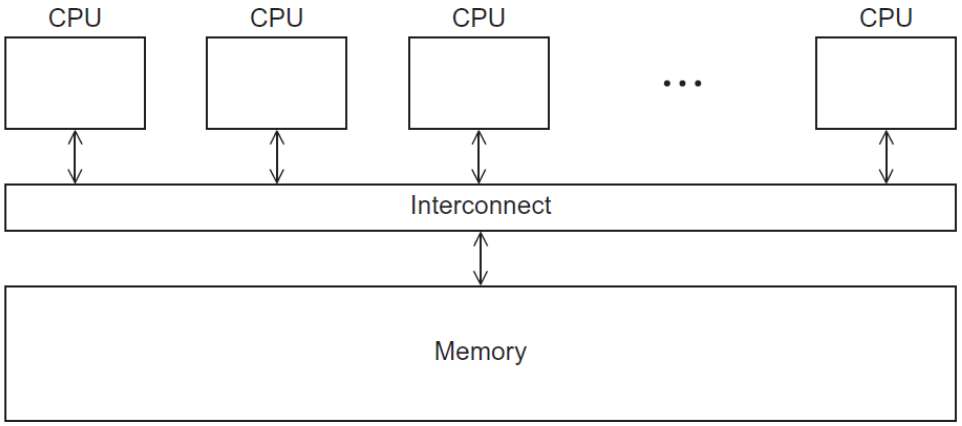
- Invited speakers from Oak Ridge National Lab
  - Dr. **Augustus McSweeney-Ellerm** →University of Chicago & ANL
    - November 19 (18.00-19.00)
  - Dr. **Davide Salomoni** → National Center on HPC, Big Data and QC
    - December 04 (11.30-12.30)
  - Dr. **Wes Brewer** → Oak Ridge National Laboratory (US)
  - Dr. **Valentine Anantharaji** → Oak Ridge National Laboratory (US)

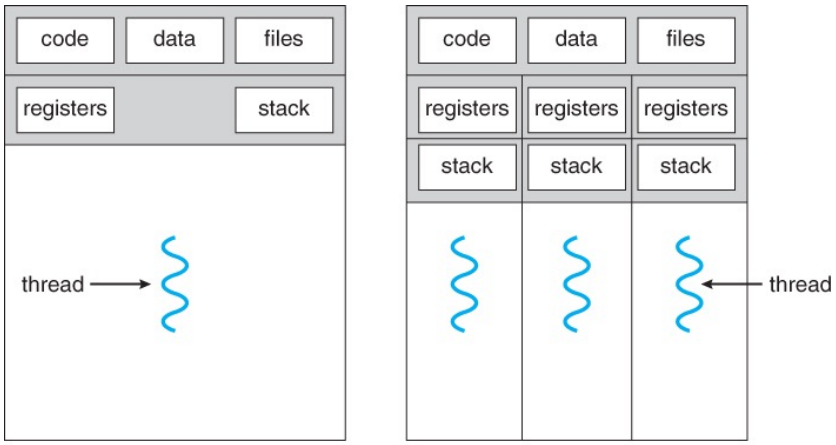




OpenMP and shared memory system

- An API for shared-memory parallel programming.
- MP = multiprocessing
- Designed for systems in which each thread or process can potentially have access to all available memory.
- System is viewed as a collection of cores or CPU's, all of which have access to main memory.





single-threaded process multithreaded process

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Hello World example

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

 printf("Hello from the serial world\n");

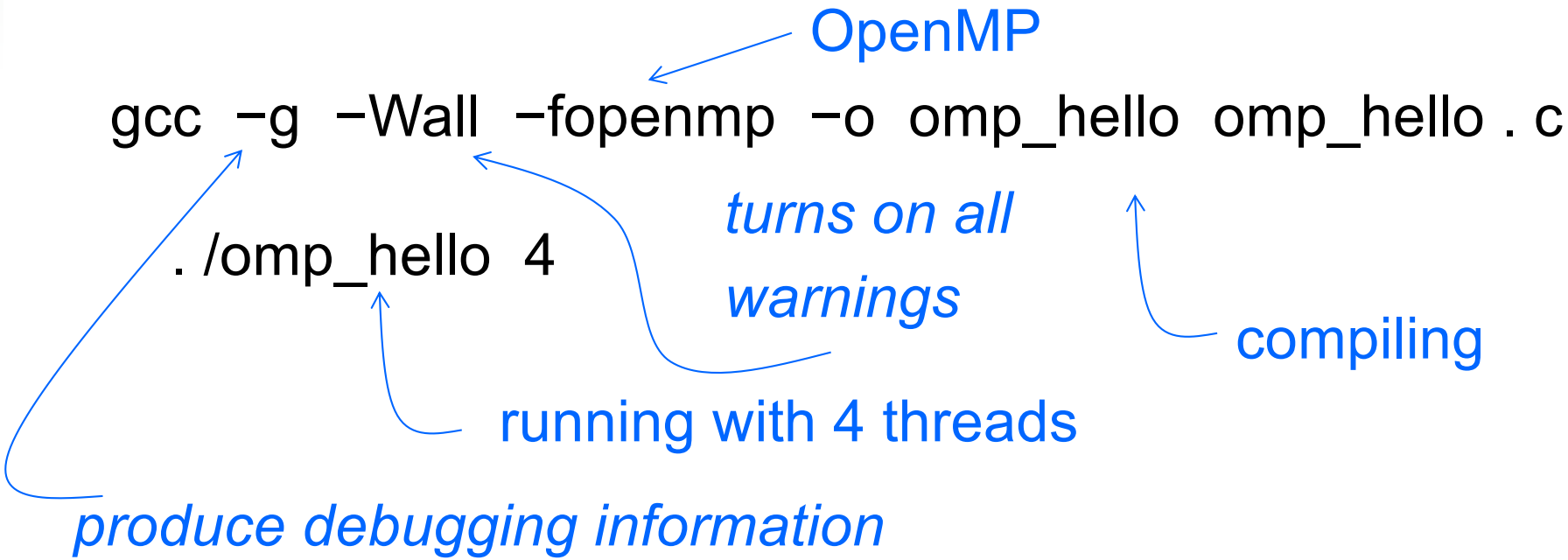
#pragma omp parallel num_threads(10)
 {
  printf("Hello World... from thread = %d\n", omp_get_thread_num());
 }

 printf("Back to the serial world\n");
 return 0;
}
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP key aspects

- Shared memory parallel programming model
- Pragma based approach and compiler support (_OPENMP macro)
- Incremental parallelism
- Fork-Join model
- Pragma omp -> directives -> clauses
- # pragma omp parallel



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

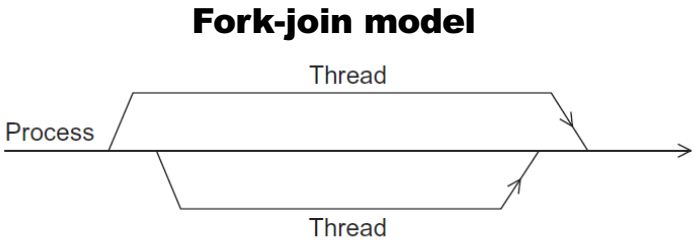
Pragmas, directives and clauses

pragmas

- Special preprocessor instructions.
- Typically added to a system to allow behaviors that aren't part of the basic C specification.
- Compilers that don't support the pragmas ignore them.

pragma omp parallel

- Most basic parallel directive.
- The number of threads that run the following structured block of code is determined by the run-time system.



pragma omp parallel **num_threads** (thread_count)

Copyright © 2010, Elsevier Inc. All rights Reserved

Of note…

- There may be system-defined limitations on the number of threads that a program can start.
- The OpenMP standard doesn't guarantee that this will actually start thread_count threads.
- Most current systems can start hundreds or thousands of threads.
- Unless we're trying to start a lot of threads, we will almost always get the desired number of threads.

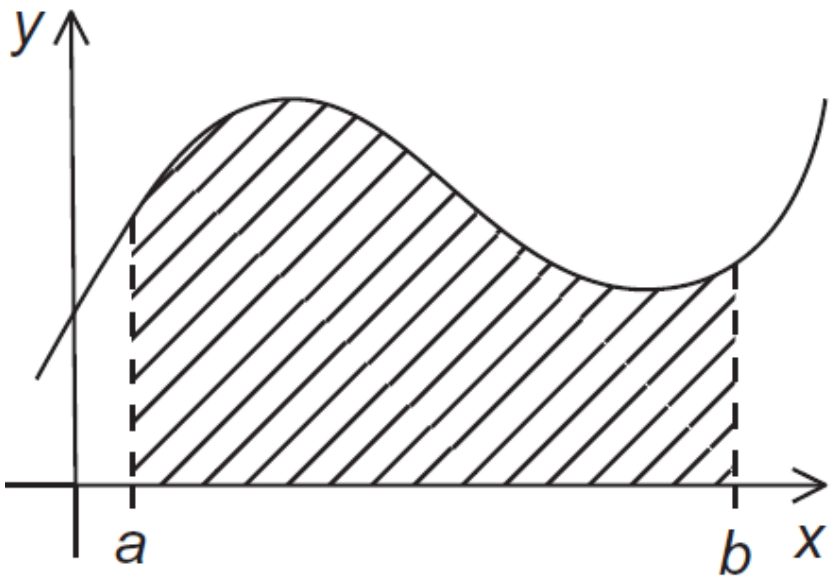
Copyright © 2010, Elsevier Inc. All rights Reserved

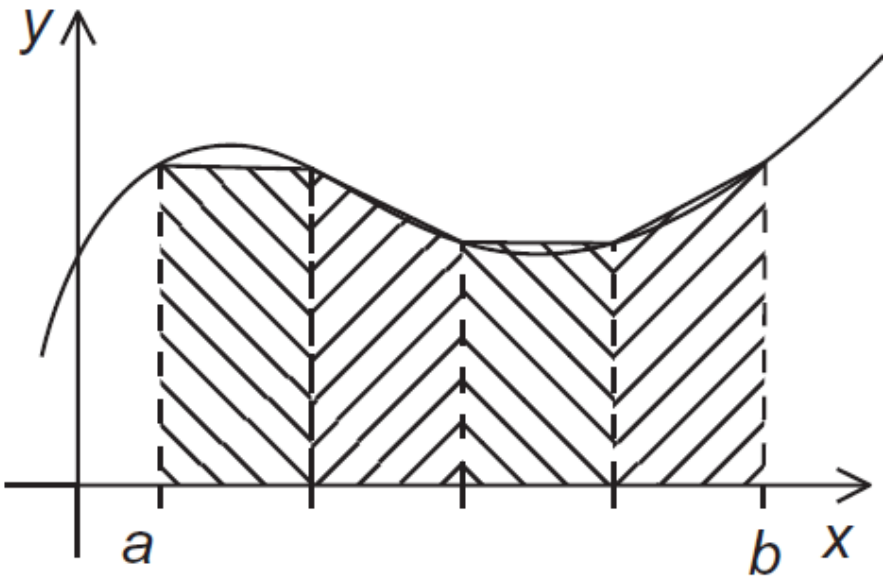
Scope of a variable

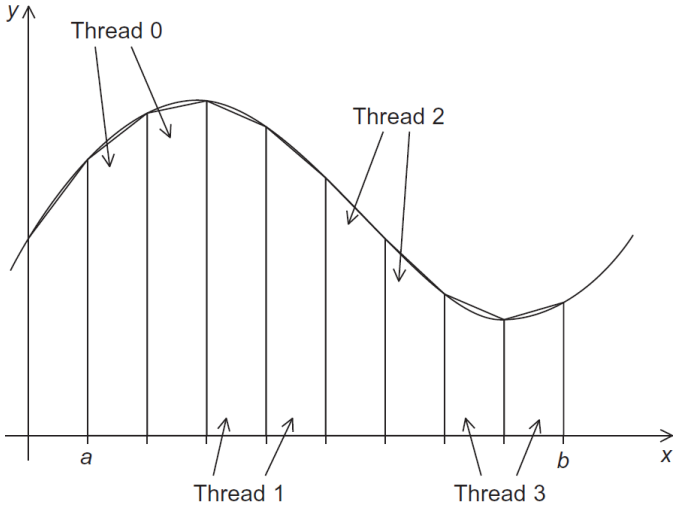
- In serial programming, the **scope** of a variable consists of those **parts of a program in which the variable can be used**.
- In OpenMP, the scope of a variable refers **to the set of threads that can access the variable in a parallel block**.
  - A variable that can be accessed by all the threads in the team has shared scope.
  - A variable that can only be accessed by a single thread has private scope.
  - The default scope for variables declared before a parallel block is shared.

Copyright © 2010, Elsevier Inc. All rights Reserved

The trapezoidal rule








```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

A First OpenMP Version

1) We identified two types of tasks:
   a) computation of the areas of individual trapezoids, and
   b) adding the areas of trapezoids.
2) There is no communication among the tasks in the first collection, but each task in the first collection communicates with task 1b.
3) We assumed that there would be many more trapezoids than cores.

- So, we aggregated tasks by assigning a contiguous block of trapezoids to each thread (and a single thread to each core).
- Critical point: global sum across the partial thread results
- Pragra omp critical can help in this scenario

```
# pragma omp critical
  global_result += my_result ;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Parallel and critical directives

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

MPI style

```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
}  /* Trap */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

High Performance Computing for Data Science

Reduction clause and parallel for directive
Lecture 15 - 05/11/2026

The reduction clause

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Going back to the code

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

We need this more complex version to add each thread's local calculation to get *global_result*.

```
void Trap(double a, double b, int n, double* global_result_p);
```

Although we'd prefer this.

```
double Trap(double a, double b, int n);
```



```
global_result = Trap(a, b, n);
```


Copyright © 2010, Elsevier Inc. All rights Reserved

If we use this, there's no critical section!

```
double Local_trap(double a, double b, int n);
```

If we fix it like this…

```
    global_result = 0.0;
#   pragma omp parallel num_threads(thread_count)
    {
#       pragma omp critical
        global_result += Local_trap(double a, double b, int n);
    }
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

We can avoid this problem by declaring a private variable inside the parallel block and moving the critical section after the function call.

```
    global_result = 0.0;
#   pragma omp parallel num_threads(thread_count)
    {
       double my_result = 0.0;  /* private */

       my_result += Local_trap(double a, double b, int n);
#      pragma omp critical
       global_result += my_result;
    }
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

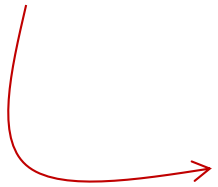
Reduction operators

- A reduction operator is a binary operation (such as addition or multiplication).
- A reduction is a computation that repeatedly applies the same reduction operator to a sequence of operands in order to get a single result.
- All of the intermediate results of the operation should be stored in the same variable: the ***reduction variable***.

Copyright © 2010, Elsevier Inc. All rights Reserved

A reduction clause can be added to a parallel directive.

```
reduction(<operator>: <variable list>)
```



+, *, -, &, |, ^, &&, ||

```
    global_result = 0.0;
#   pragma omp parallel num_threads(thread_count) \
       reduction(+: global_result)
    global_result += Local_trap(double a, double b, int n);
```


Copyright © 2010, Elsevier Inc. All rights Reserved

High Performance Computing for Data Science

Reduction clause and parallel for directive
Lecture 15 - 05/11/2026

The parallel for directive

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

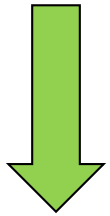
Parallel for

- **Forks a team of threads** to execute the following structured block.
- However, **the structured block** following the parallel for directive **must be a for loop**.
- Furthermore, with the *parallel for directive* the system *parallelizes the for loop by dividing the iterations* of the loop *among the threads*.

Copyright © 2010, Elsevier Inc. All rights Reserved

```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
   approx += f(a + i*h);
approx = h*approx;
```





```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
} /* Trap */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
} /* main */
```

```
   h = (b-a)/n;
   approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx)
   for (i = 1; i <= n-1; i++)
      approx += f(a + i*h);
   approx = h*approx;
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Legal forms for parallelizable for statements

```
for ( index = start ; index < end    ; index++   )
                    ; index <= end   ; ++index   )
                    ; index >= end   ; index--   )
                    ; index > end    ; --index   )
                                     ; index += incr
                                     ; index -= incr
                                     ; index = index + incr
                                     ; index = incr + index
                                     ; index = index - incr
```

Copyright © 2010, Elsevier Inc. All rights Reserved

Caveats

- The variable index must have integer or pointer type (e.g., it can't be a float).
- The expressions start, end, and incr must have a compatible type. For example, if index is a pointer, then incr must have integer type.

Copyright © 2010, Elsevier Inc. All rights Reserved

Caveats

- The expressions start, end, and incr must not change during execution of the loop.
- During execution of the loop, the variable index can only be modified by the "increment expression" in the for statement.

Copyright © 2010, Elsevier Inc. All rights Reserved

Data dependencies

```
fibo[ 0 ] = fibo[ 1 ] = 1;
for (i = 2; i < n; i++)
    fibo[ i ] = fibo[ i – 1 ] + fibo[ i – 2 ];
```



note 2 threads



```
fibo[ 0 ] = fibo[ 1 ] = 1;
# pragma omp parallel for num_threads(2)
for (i = 2; i < n; i++)
    fibo[ i ] = fibo[ i – 1 ] + fibo[ i – 2 ];
```



1 1 2 3 5 8 13 21 34 55

this is correct



but sometimes we get this

1 1 2 3 5 8 0 0 0 0

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

What happened?



1. OpenMP compilers don't check for dependences among iterations in a loop that's being parallelized with a parallel for directive.
2. A loop in which the results of one or more iterations depend on other iterations cannot, in general, be correctly parallelized by OpenMP.

Copyright © 2010, Elsevier Inc. All rights Reserved

Estimating π

$$\pi = 4\left[1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right] = 4\sum_{k=0}^{\infty}\frac{(-1)^k}{2k+1}$$

```
double factor = 1.0;
double sum = 0.0;
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP solution #1

loop dependency

```
        double factor = 1.0;
        double sum = 0.0;
#       pragma omp parallel for num_threads(thread_count) \
            reduction(+:sum)
        for (k = 0; k < n; k++) {
            sum += factor/(2*k+1);
            factor = -factor;
        }
        pi_approx = 4.0*sum;
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP solution #2

```
        double sum = 0.0;
#       pragma omp parallel for num_threads(thread_count) \
            reduction(+:sum) private(factor)
        for (k = 0; k < n; k++) {
            if (k % 2 == 0)
               factor = 1.0;
            else
               factor = -1.0;
            sum += factor/(2*k+1);
        }
```

Insures factor has private scope.

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

The default clause

- Lets the programmer specify the scope of each variable in a block.

```
default(none)
```

- With this clause the compiler will require that we specify the scope of each variable we use in the block and that has been declared outside the block.

Copyright © 2010, Elsevier Inc. All rights Reserved

The default clause

```
        double sum = 0.0;
#       pragma omp parallel for num_threads(thread_count) \
           default(none) reduction(+:sum) private(k, factor) \
           shared(n)
        for (k = 0; k < n; k++) {
           if (k % 2 == 0)
              factor = 1.0;
           else
              factor = -1.0;
           sum += factor/(2*k+1);
        }
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Scoping clauses list

- **default**
- **shared**
- **private**
- **reduction**
- firstprivate
- lastprivate

Scoping clauses: first/lastprivate

- At the start of a parallel for, **firstprivate** initializes each thread's copy of a private variable to the value of the master's copy
  - Once per thread
  - Performance implications with respect to private
  - one-to-many

- At the end of a parallel for, **lastprivate** writes back to the master's copy the value contained in the private copy belonging to the thread that executed the "sequentially" last iteration of the loop
  - one-to-one (not many to one!)

Lexical (static) and dynamic extent

- We define the **lexical (or static) extent** of a parallel region as the code that is lexically within the parallel/end parallel directive

- We define **dynamic extent** of a parallel region to include not only the code that is directly between the parallel/end parallel directive (or static extent), but also to include all the code in subprograms that are invoked either directly or indirectly from within the parallel region.

Default scoping rules in C

```
void caller(int a[], int n)
{
    int i, j, m = 3;

    #pragma omp parallel for
    for (i = 0; i < n; i++) {
        int k = m;

        for (j = 1; j ≤ 5; j++)
            callee(&a[i], &k, j);
    }
}

extern int c;

void callee(int *x, int *y, int z)
{
    int ii;
    static int cnt;

    cnt++;
    for (ii = 0; ii < z; i++)
        *x = *y + c;
}
```

Exercise: default scoping rules in C

Variable
a
n
i
j
m
k
x
*x
y
*y
z
c
ii
cnt

For each variable in the list, please provide:

- *Scope (shared/private)*
- *Is use safe? (Yes/No)*
- *Reason for scope*

Homework parallel/parallel for

- “Hello World” (parallel directive)
- Trapezoidal rule (parallel + critical)
- Trapezoidal rule (parallel for + reduction)
- Estimating π (parallel for + reduction)
- Try to run it on the cluster…

Copyright © 2010, Elsevier Inc. All rights Reserved

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLd9xlKlHkeBM2x_JgWyA0qQu6mOBI5A/viewform